SQL Projects

Build a Portfolio of Real SQL Analyses

LEARNING OUTCOME

By the end of this module you will have 4 complete, portfolio-ready SQL projects: an e-commerce analysis, employee HR analytics, financial reporting system, and a Twitter-style social network schema — all with production-quality queries.

Skills You Will Gain
• Design and analyse an e-commerce database
• Write HR analytics queries for employee data
• Build a financial reporting system with CTEs
• Design a social network schema
• Write interview-ready SQL queries
• Present findings in a professional analysis

■ **SQL projects demonstrate that you can apply skills to real business problems. One well-documented SQL portfolio on GitHub attracts more data job interviews than any certificate alone.**

PROJECT 1

E-Commerce Sales Analysis

```
-- DATABASE SCHEMA -- customers(customer_id, name, city, signup_date, is_premium) --
orders(order_id, customer_id, order_date, status, total_amount) -- order_items(item_id, order_id,
product_id, quantity, unit_price) -- products(product_id, name, category, price, cost) -- ❶ Revenue
by month with MoM growth WITH monthly_rev AS ( SELECT DATE_FORMAT(order_date,'%Y-%m') AS month,
SUM(total_amount) AS revenue FROM orders WHERE status = 'delivered' GROUP BY
DATE_FORMAT(order_date,'%Y-%m') ) SELECT month, revenue, LAG(revenue) OVER (ORDER BY month) AS
prev_month, ROUND(100.0*(revenue-LAG(revenue) OVER (ORDER BY month)) /NULLIF(LAG(revenue) OVER
(ORDER BY month),0),2) AS mom_growth_pct FROM monthly_rev ORDER BY month; -- ❷ Customer Lifetime
Value + Segmentation WITH clv AS ( SELECT c.customer_id, c.name, c.city, COUNT(o.order_id) AS
orders, SUM(o.total_amount) AS ltv, AVG(o.total_amount) AS aov,
DATEDIFF(MAX(o.order_date),MIN(o.order_date)) AS tenure_days FROM customers c LEFT JOIN orders o ON
c.customer_id=o.customer_id AND o.status='delivered' GROUP BY c.customer_id, c.name, c.city )
SELECT *, NTILE(4) OVER (ORDER BY ltv DESC) AS quartile, CASE NTILE(4) OVER (ORDER BY ltv DESC)
WHEN 1 THEN 'Champions' WHEN 2 THEN 'Loyal' WHEN 3 THEN 'At Risk' ELSE 'Inactive' END AS segment
FROM clv ORDER BY ltv DESC; -- ❸ Product Performance -- margin and return rate SELECT p.category,
p.name, SUM(oi.quantity) AS units_sold, SUM(oi.quantity*oi.unit_price) AS revenue,
SUM(oi.quantity*(oi.unit_price-p.cost)) AS gross_profit,
ROUND(100.0*SUM(oi.quantity*(oi.unit_price-p.cost)) /NULLIF(SUM(oi.quantity*oi.unit_price),0),2) AS
margin_pct, ROUND(100.0*COUNT(CASE WHEN o.status='returned' THEN 1 END) /COUNT(*),2) AS
return_rate_pct FROM products p JOIN order_items oi ON p.product_id=oi.product_id JOIN orders o ON
oi.order_id=o.order_id GROUP BY p.category, p.product_id, p.name ORDER BY gross_profit DESC;
```

PROJECT 2

HR Analytics and Financial Reporting

```
-- ===== HR ANALYTICS ===== -- employees(emp_id, name, dept_id, manager_id, salary, hire_date,
grade) -- departments(dept_id, dept_name, location) -- ❶ Salary analysis by department SELECT
d.dept_name, COUNT(e.emp_id) AS headcount, AVG(e.salary) AS avg_salary, MIN(e.salary) AS
min_salary, MAX(e.salary) AS max_salary, SUM(e.salary) AS payroll, STDDEV(e.salary) AS
salary_stddev FROM departments d JOIN employees e ON d.dept_id = e.dept_id GROUP BY d.dept_id,
d.dept_name ORDER BY payroll DESC; -- ❷ Find employees paid more than their department average
SELECT e.name, e.salary, dept_avg.avg_salary, e.salary - dept_avg.avg_salary AS diff_from_avg FROM
employees e JOIN ( SELECT dept_id, AVG(salary) AS avg_salary FROM employees GROUP BY dept_id )
dept_avg ON e.dept_id = dept_avg.dept_id WHERE e.salary > dept_avg.avg_salary ORDER BY
diff_from_avg DESC; -- ❸ Org chart (recursive CTE) WITH RECURSIVE tree AS ( SELECT emp_id, name,
manager_id, salary, 0 AS lvl, CAST(name AS CHAR(200)) AS path FROM employees WHERE manager_id IS
NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, e.salary, t.lvl+1, CONCAT(t.path,' >
',e.name) FROM employees e JOIN tree t ON e.manager_id=t.emp_id ) SELECT REPEAT(' ',lvl)||name AS
hierarchy, salary, path FROM tree ORDER BY path; -- ===== FINANCIAL REPORTING ===== --
transactions(txn_id, account_id, txn_date, amount, type, category) -- ❹ Running balance per account
SELECT account_id, txn_date, amount, type, SUM(CASE WHEN type='credit' THEN amount ELSE -amount
END) OVER (PARTITION BY account_id ORDER BY txn_date, txn_id) AS running_balance FROM transactions
ORDER BY account_id, txn_date; -- ❺ Category spending vs budget SELECT t.category, SUM(t.amount) AS
actual_spend, b.budget_amount AS budget, SUM(t.amount)-b.budget_amount AS variance,
ROUND(100.0*SUM(t.amount)/b.budget_amount,1) AS pct_of_budget FROM transactions t JOIN budget b ON
t.category = b.category WHERE t.type = 'debit' AND YEAR(t.txn_date) = 2024 GROUP BY t.category,
b.budget_amount HAVING SUM(t.amount) > b.budget_amount -- over budget only ORDER BY variance DESC;
```

PROJECT 3

Social Network Schema & Queries

```
-- SOCIAL NETWORK SCHEMA CREATE TABLE users ( user_id INT PRIMARY KEY, username VARCHAR(30) UNIQUE, bio TEXT, created_at TIMESTAMP DEFAULT NOW() ); CREATE TABLE follows ( follower_id INT, following_id INT, followed_at TIMESTAMP DEFAULT NOW(), PRIMARY KEY (follower_id, following_id), FOREIGN KEY (follower_id) REFERENCES users(user_id), FOREIGN KEY (following_id) REFERENCES users(user_id) ); CREATE TABLE posts ( post_id INT PRIMARY KEY, user_id INT, content TEXT, created_at TIMESTAMP DEFAULT NOW(), FOREIGN KEY (user_id) REFERENCES users(user_id) ); CREATE TABLE likes ( user_id INT, post_id INT, liked_at TIMESTAMP DEFAULT NOW(), PRIMARY KEY (user_id, post_id) ); -- ❶ Feed: posts from users you follow SELECT p.post_id, u.username, p.content, p.created_at, COUNT(l.user_id) AS like_count FROM posts p JOIN users u ON p.user_id = u.user_id JOIN follows f ON f.following_id = p.user_id LEFT JOIN likes l ON p.post_id = l.post_id WHERE f.follower_id = 1 -- current user AND p.created_at >= NOW() - INTERVAL '7 days' GROUP BY p.post_id, u.username, p.content, p.created_at ORDER BY p.created_at DESC LIMIT 20; -- ❷ Mutual followers SELECT u.username AS mutual_friend FROM users u JOIN follows f1 ON f1.following_id=u.user_id AND f1.follower_id=1 JOIN follows f2 ON f2.following_id=u.user_id AND f2.follower_id=2 WHERE u.user_id NOT IN (1,2); -- ❸ Trending posts (likes per hour) SELECT p.post_id, u.username, p.content, COUNT(l.user_id) AS likes, TIMESTAMPDIFF(HOUR, p.created_at, NOW()) AS age_hours, COUNT(l.user_id) / NULLIF(TIMESTAMPDIFF(HOUR,p.created_at,NOW()),0) AS likes_per_hour FROM posts p JOIN users u ON p.user_id=u.user_id LEFT JOIN likes l ON p.post_id=l.post_id WHERE p.created_at >= NOW() - INTERVAL '48 hours' GROUP BY p.post_id, u.username, p.content, p.created_at ORDER BY likes_per_hour DESC LIMIT 10;
```

SECTION 4

SQL Interview Preparation

Classic Interview Question	Required Concept
Find the Nth highest salary	DENSE_RANK() or subquery with LIMIT OFFSET
Employees earning more than manager	Self JOIN or correlated subquery
Duplicate records in a table	GROUP BY id HAVING COUNT(*) > 1
Running total / cumulative sum	SUM() OVER (ORDER BY date)
Month-over-month growth %	LAG() window function
Customers with no orders	LEFT JOIN WHERE id IS NULL or NOT EXISTS
Most purchased product per city	RANK() OVER (PARTITION BY city ORDER BY cnt DESC)
Missing dates in time series	Generate date sequence, LEFT JOIN to find gaps
Consecutive login days	Date arithmetic with ROW_NUMBER
Pivot rows to columns	CASE WHEN / conditional aggregation

■ PRACTICE Q & A

Q1. How do you find duplicate records in a table?

> ***A:*** *SELECT col1, col2, COUNT(*) AS cnt FROM table GROUP BY col1, col2 HAVING COUNT(*) > 1. To delete duplicates keeping one: use ROW_NUMBER() OVER (PARTITION BY col1,col2 ORDER BY id) and delete where row_num > 1.*

Q2. How do you find the second highest salary?

> ***A:*** *SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees). Or with DENSE_RANK: SELECT salary FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM employees) ranked WHERE rnk = 2.*

Q3. How would you design a schema for a simple e-commerce site?

> ***A:*** *customers(id,name,email,city) -- orders(id,customer_id,date,status,total) -- products(id,name,category,price,cost) -- order_items(id,order_id,product_id,qty,price). Foreign keys: orders.customer_id -> customers.id, order_items.order_id ->*

orders.id.

Q4. What indexes would you create for the orders table?

***A:** idx on customer_id (for customer order lookups), idx on order_date (for date range filters), idx on status (if filtered frequently, only if high cardinality), composite idx on (customer_id, order_date DESC) for customer order history sorted by date.*

Q5. How do you calculate a 7-day moving average in SQL?

***A:** AVG(col) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW). This includes current row plus 6 preceding = 7 rows total.*

■ SQL Projects Cheat Sheet	
CTE for multi-step analysis	`WITH step1 AS (...), step2 AS (...) SELECT`
MoM growth	`LAG(revenue) OVER (ORDER BY month)`
Customer segmentation	`NTILE(4) OVER (ORDER BY ltv DESC)`
Running balance	`SUM(amount) OVER (PARTITION BY account ORDER BY date)`
Anti-join	`LEFT JOIN WHERE right.id IS NULL`
Recursive org chart	`WITH RECURSIVE cte AS (anchor UNION ALL ...)`
Nth highest salary	`DENSE_RANK() OVER (ORDER BY salary DESC)`
Duplicates	`GROUP BY key HAVING COUNT(*) > 1`
Pivot	`SUM(CASE WHEN cat='A' THEN amt ELSE 0 END)`
Top N per group	`RANK() OVER (PARTITION BY grp ORDER BY val)`
Trending score	`likes / NULLIF(age_hours, 0)`
Margin %	`(revenue-cost)/NULLIF(revenue,0)*100`